Faster expert loading in colibri

Results report for Mr Shravan · Mostafa Elhadidy · 27 October 2026 · all figures generated from `benchmarks/expert_cache_research/` by `docs/report_src/build_report.py`

Summary. colibri runs large **Mixture-of-Experts** (MoE) language models on ordinary computers. Each layer of such a model has many small sub-networks ("experts"), of which only a few run for each token. The experts do not fit in RAM, so colibri keeps a small cache of them and reads the rest from disk as they are needed. We measured that on this machine **the time goes to loading experts from disk, not to computing with them**. Two changes attack that loading:

- **Parallel loading** reads a layer's missing experts 8 at a time instead of one at a time.
- **"Waves"** process a prompt expert by expert instead of token by token, so each expert is read once instead of once per token.

Measured on the 514-token prompt, waves cut the **time to first token** from 376 s to 30.0 s (-92%) with a 12-expert cache, and from 221 s to 47.0 s with a 32-expert cache. Disk reads fall from 283 GB to 35 GB. Parallel loading had already made the first token 39–68% faster. The model's output is **bit-for-bit identical** to the original program in every one of the 172 runs of the modified program.

1. Setup

- **Model:** Qwen3-Coder-30B-A3B: 48 layers × 128 experts, 8 experts per token per layer. Experts are 4-bit, 2.53 MiB each, 15.2 GiB in total (19 GB on disk). It runs on colibri's `qwen36` engine.
- **Machine:** Intel i9-14900HX laptop, NVMe SSD, Windows 11 with WSL2 (a Linux virtual machine) limited to 12 GB of the host's 16 GB RAM. The experts alone are 15.2 GiB, more than the VM can hold next to everything else, so they cannot all be in RAM. The GPU tier needs every expert in RAM and is therefore unusable, so everything runs on the CPU.
- **Cache size ("cap"):** the number of experts kept in RAM per layer. We test **cap 32** (peak memory 6.2 GB) and **cap 12** (3.9 GB, simulating a small machine).
- **Why Qwen3-Coder stands in for GLM-5.3-Flash.** GLM-5.3-Flash, the real target, is a 321-billion-parameter MoE (195 GB) that cannot run in 12 GB. Qwen3-Coder has the same structure (routed experts streamed from disk through a per-layer cache) and fits, so the mechanisms can be measured here and then ported (§7).
- **How it was measured:** every run is **cold**: a fresh process, with the operating system's file cache emptied first, so every expert really comes from disk. The variants run back to back and alternate their order (A B B A), 3 repetitions each. A difference counts only if its 95% **bootstrap confidence interval** (an interval estimated by resampling the measured runs) excludes zero; otherwise it is marked *n.s.* (not significant). The disk is a virtual disk on the laptop and is noisy: the per-run probe (median 9.8 ms per expert read) varied several-fold, which is why only back-to-back comparisons are used.

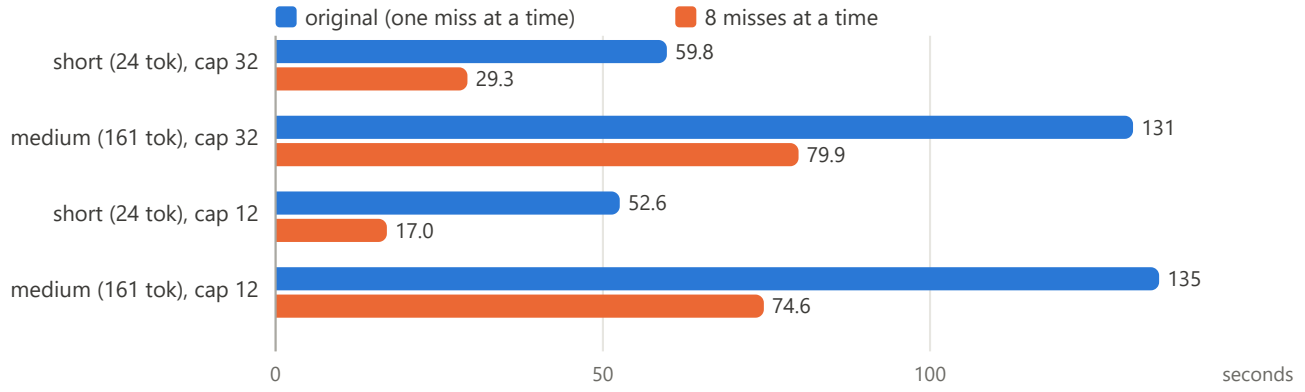
2. Parallel loading (MISS_QD=8)

What it does. When a layer needs experts that are not in the cache (**cache misses**), the original code read them one after another. Each read waits for the disk, and the disk was mostly idle: it delivers about 2.8–4.0× more at 4–8 reads in flight than at one. The change first decides all of a layer's hits and misses in one pass (exactly the decisions the original makes), then reads the misses 8 at a time.

Why it helps. The waiting overlaps. Nothing about which experts run changes.

prompt, cache	time to first token	change [95% CI]	tokens/s, end to end	change
short (24 tok), cap 32	59.8 → 29.3 s	**-51%** [-58, -42]	0.31 → 0.52	**+66%** [+32, +111]
medium (161 tok), cap 32	131 → 79.9 s	**-39%** [-48, -26]	0.20 → 0.31	**+59%** [+25, +96]
short (24 tok), cap 12	52.6 → 17.0 s	**-68%** [-82, -37]	0.46 → 0.85	**+84%** [+32, +210]
medium (161 tok), cap 12	135 → 74.6 s	**-45%** [-46, -43]	0.18 → 0.32	**+80%** [+70, +90]

Parallel loading: time to first token (s), original vs. MISS_QD=8



Mean of 3 cold runs per bar. The medium prompt is 161 tokens and the short one 24; the 514-token prompt was not part of this comparison.

---

3. Waves: reading each expert once per prompt

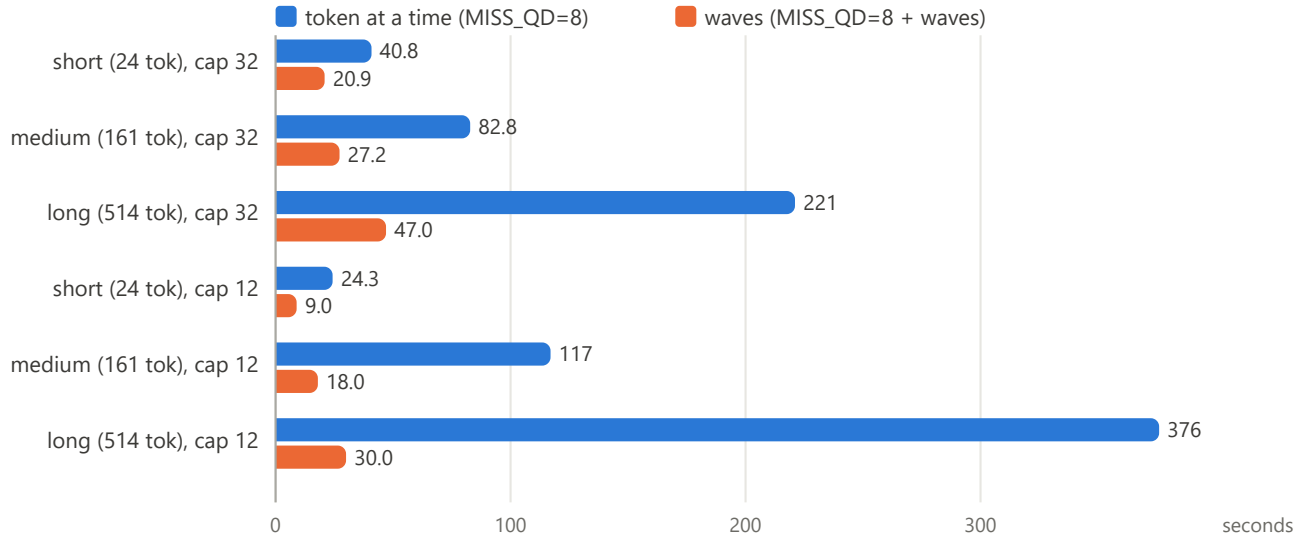
The problem. A prompt of S tokens needs S × 8 experts per layer. That is far more than the cache holds, so the original falls back to processing *one token at a time*. Every token re-reads experts the previous tokens evicted, even though a layer only has 128 distinct experts. The 514-token prompt read 116,777 experts (283 GB) before its first output token at cap 12.

What it does.

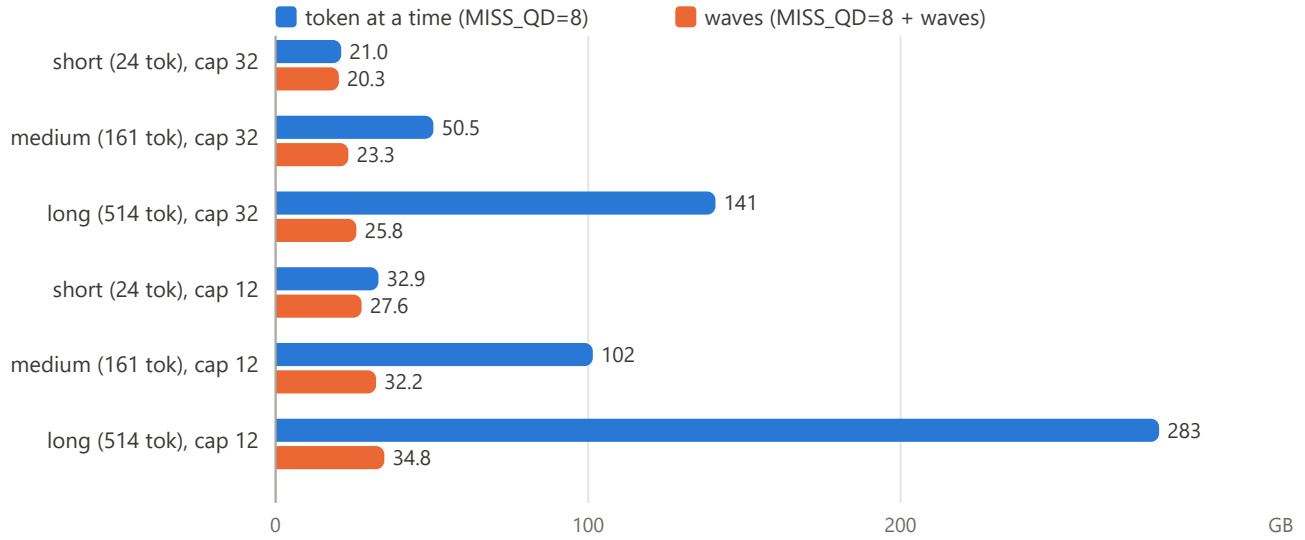
1. The router (the small network that picks each token's 8 experts) runs for every prompt token first.
2. The distinct experts are processed in "waves" no larger than the cache, starting with the ones already in RAM.
3. While a wave is in RAM, every token that needs those experts is computed.
4. Each token's 8 results are added in the original order with the original weights, so the output is bit-identical.

prompt, cache	first token, s	change [95% CI]	GB read	experts read	tokens/s, end to end	decode tokens/s
short (24 tok), cap 32	40.8 → 20.9	**-49%** [-68, -7]	21.0 → 20.3	6,629 → 6,289	+12% *n.s.*	-15% *n.s.*
medium (161 tok), cap 32	82.8 → 27.2	**-67%** [-74, -58]	50.5 → 23.3	18,843 → 7,432	**+72%** [+35, +110]	**-28%** [-48, -1] **worse**
long (514 tok), cap 32	221 → 47.0	**-79%** [-85, -71]	141 → 25.8	56,703 → 8,363	**+186%** [+131, +250]	-8% *n.s.*
short (24 tok), cap 12	24.3 → 9.0	**-63%** [-75, -44]	32.9 → 27.6	11,658 → 9,377	+58% *n.s.*	+20% *n.s.*
medium (161 tok), cap 12	117 → 18.0	**-85%** [-89, -77]	102 → 32.2	40,440 → 11,203	**+252%** [+102, +452]	+51% *n.s.*
long (514 tok), cap 12	376 → 30.0	**-92%** [-94, -90]	283 → 34.8	116,777 → 12,261	**+571%** [+407, +794]	+48% *n.s.*

Waves: time to first token (s)



Waves: expert bytes read from disk per run (GB)



Baseline here is MISS_QD=8 (§2), so these gains come on top of parallel loading. "Decode" is generating the 31 tokens after the first: it reads only a few experts per token and is unchanged within the noise in 5 of 6 cells (**-28%** [-48, -1] worse in the sixth, medium (161 tok), cap 32; see §7).

Planning window. How many tokens are planned into waves at once matters: a bigger window means fewer re-reads. 514-token prompt, mean of 2 cold runs:

tokens planned at once	cap 32: GB read	first token, s	cap 12: GB read	first token, s
32	86.3	239	121.0	221
64	64.8	231	87.5	154
128	47.0	157	62.3	115
256	33.8	115	45.9	56
whole prompt (514)	25.9	68	35.1	53

4. Phase 0: what the "page faults" were

The first study counted ~3.0 million **minor page faults** per short run, of the same order as one fault per 4 KiB page of every expert loaded. A page fault is the operating system mapping a fresh 4 KiB page of memory the first time a program touches it. That looked like a per-miss overhead. We timed every step of a miss and counted its faults. They are **not per miss**:

- 2.03 M happen while the model loads, before any token;
- 1.00 M are the expert cache's memory being touched for the first time, i.e. 48 layers × 32 slots × 648 pages per expert;
- decode, which is all misses into reused memory, takes **0**.

The real per-miss costs are:

- a **second disk read** for each expert's scale factors, which the file stores apart from its weights (2.0 ms);
- rewriting an evicted cache slot takes 3.0 ms, against 0.28 ms for fresh memory, with no fault at all.

The cause of the second is outside the program. During a run the Windows host fell to 4 MB of free RAM and pushed the virtual machine down to 876 MB of real memory while the program held 4,157 MB. The least recently used cache slot is exactly the memory the host has paged out, so rewriting it waits for the host. A huge-page memory arena (`COLI_Q36_SLOT_ARENA`) removes the first-touch faults (-11% to -34%), but 17 of 18 timing comparisons are not significant, so it stays off.

5. Grouped kernel: correct but not worth enabling

With waves, each expert computes many tokens at once. The grouped kernel processes 4 tokens per pass over the expert's weights (each weight block is loaded and unpacked once instead of four times). Each token's arithmetic keeps its exact order, so this turned out **exact**, not approximate: identical output, maximum logit difference 0.

prompt, cache	expert compute time in prefill	change in first-token time
medium (161 tok), cap 32	4.7 → 4.9 s	0% *n.s.*
long (514 tok), cap 32	10.1 → 8.4 s	0% *n.s.*
medium (161 tok), cap 12	3.7 → 3.3 s	**+58%** [+11, +126] worse
long (514 tok), cap 12	8.5 → 7.8 s	-3% *n.s.*

It saves 17% of compute on the long prompt at cap 32. But compute is only ~10 s of a 49.8 s first token, so the first token does not measurably change. The one "worse" cell traces to disk time, which the kernel does not touch. It is worth re-testing on a machine where loading is fast.

6. Correctness

- **Identical output.** In every run the program wrote its final scores over the whole vocabulary (the **logits**, 151,936 numbers) to a file. We compared their SHA-256 hash and the generated text against the **frozen original program**. 170 runs of the modified program across all variants, prompt lengths and cache sizes: **0 differences**. After the final change (last-use wave order made the default) 2 more runs: **0 differences**.
- **Unit test.** 13 configurations of waves vs. token-at-a-time give bit-identical results, every (token, expert) pair is computed exactly once, and no expert is read twice in a layer pass.
- **Same cache decisions.** The cache decisions are deterministic: hit and miss counts are identical across repetitions of each variant.

7. What didn't help or isn't proven

- **Huge-page arena** (§4): fewer faults, no measurable speed-up, +1–3% memory.
- **Decode speed** is not proven to change: 3 repetitions cannot resolve it under this host's noise (decode confidence intervals up to ±95 percentage points wide).
- **Wave order.** In first-seen order, waves leave a slightly worse set of experts cached for decode at cap 32 (decode misses 2,532 vs 1,870 on the medium prompt). Ordering by last use makes them equal to the original in all 6 cells (now the default), but its timing effect was mixed and is not claimed.
- **Imperfect cold start.** 26 of 176 cold runs started with up to 4.3 MB (of 19 GB) still cached (the rule is exactly 0): at most ~1.6 experts' worth, which is negligible.
- **Not measured:** server (warm) mode, the predictive prefetch (PILOT) with waves, the GPU paths.

8. Can this apply to GLM-5.3-Flash?

What `glm53.c` already does (from reading the code): it reads a layer's missing experts in parallel, and its prompt processing is already expert by expert, inside a **128-token window** (`GLM53_PREFILL_CHUNK`).

What would carry over:

1. **Cached experts first.** Today it walks a window's experts in first-seen order, so an expert already in RAM can be evicted before its turn and read again.
2. **A larger planning window.** 128 tokens means an expert used throughout a long prompt is re-read in every window.
3. **A fixed-order sum.** glm53 adds expert results in processing order, so reordering would change the last bits of the output. Keeping per-(token, expert) results and adding them in a fixed order keeps it exact, as done here.
4. **Last-use order**, so generation starts with the right experts cached.

> **Expected effect: an estimate from code analysis and the Qwen measurement, not a GLM measurement.** On Qwen with waves, a 128-token window read 1.8× (cap 32) and 1.8× (cap 12) the bytes of whole-prompt planning for the 514-token prompt. If GLM behaves alike, a larger window should cut its prefill expert reads by roughly 44–45% on prompts of ~500 tokens, more on longer prompts, and nothing on prompts under 128 tokens. Cached-first mainly helps when windows or requests share experts. GLM-5.3 has different expert counts, sizes and routing, so these numbers **must be measured on the cloud instance** before any claim.

---

9. Next steps

1. Port cached-first waves, the fixed-order sum and a larger window to `glm53.c`; measure on the cloud instance with the same cold, alternating protocol and the same bit-exactness check.
2. Re-measure decode on a host that is not short of memory (WSL limited to what the host can back), with ≥ 6 repetitions.
3. Merge or overlap the scale read with the weight read: one disk read per expert instead of two.
4. Then decide whether waves should be on by default (it is off, as the project's rules require).

Full reports: `docs/expert_cache_research.md` (part 1), `docs/expert_cache_research_2.md` (part 2), `docs/STATUS.md`. Raw rows: `benchmarks/expert_cache_research/{ab2,chunk,grouped,order,phase0,final_check}/`. Median host free RAM before runs: 1144 MB.